Chapter 6: Web Application Deployment and Modern Web Trends

Course: Web Application Programming (ENCT 302) **Program:** BE Computer Engineering | Tribhuvan University
College: National College of Engineering

Table of Contents

1. Full-Stack Web Development
2. Testing and Quality Assurance
3. DevOps and CI/CD
4. Deployment Platforms and Docker
5. Progressive Web Applications (PWA)
6. Responsive Web Design
7. Usability Principles
8. Modern Web Architecture Trends
9. Model Exam Questions & Answers

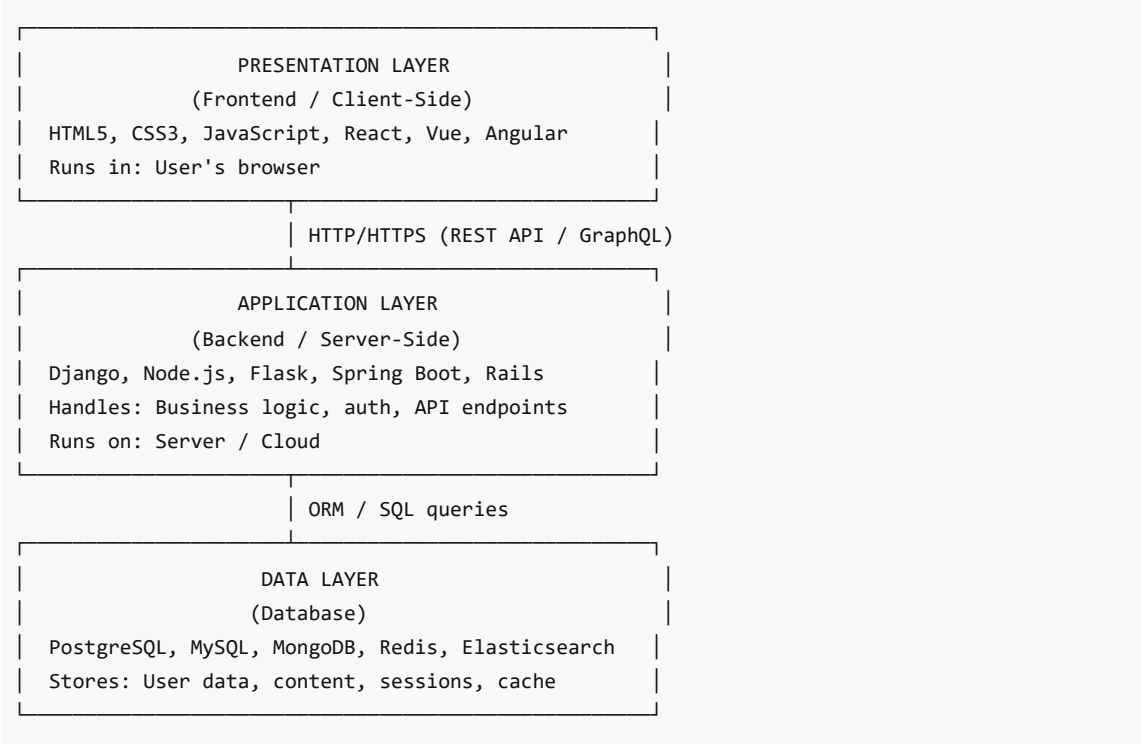
---

1. Full-Stack Web Development

What is Full-Stack Development?

A **full-stack developer** works on all layers of a web application — from the user interface visible in the browser, to the server-side logic, to the database. "Full stack" means the complete technology stack.

The Three Layers



Frontend Technologies

Technology	Purpose
HTML5	Structure and semantic content
CSS3	Styling, layout (Flexbox, Grid), animations
JavaScript (ES6+)	Interactivity, DOM manipulation, async requests
React.js	Component-based UI library (Meta/Facebook)
Vue.js	Progressive framework, easy learning curve
Angular	Full framework by Google, TypeScript-based
Webpack/Vite	Module bundlers, build tools
TypeScript	Typed superset of JavaScript

Backend Technologies

Technology	Language	Best For
Django	Python	Rapid development, batteries-included
Node.js + Express	JavaScript	Real-time, microservices, same language as frontend
Flask	Python	Lightweight APIs, microservices
Spring Boot	Java	Enterprise, banking, large-scale systems
Ruby on Rails	Ruby	Startup speed, convention over configuration
FastAPI	Python	High-performance async APIs, auto-documentation
Laravel	PHP	Content-heavy sites, rapid development

How Frontend and Backend Integrate

Traditional (Server-Side Rendering):

```
Browser request → Django view → Query DB → Render HTML template → Send full HTML
```

Modern SPA + REST API:

```
1. Browser loads index.html (empty shell)
2. React/Vue app loads in browser
3. JavaScript makes API calls: GET /api/products/
4. Backend (Django DRF) queries DB → returns JSON
5. React renders JSON data as components
6. User interactions trigger more API calls
```

Modern Full-Stack Example with React + Django:

```python
# Backend: Django REST API
# api/views.py
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Product
from .serializers import ProductSerializer

@api_view(['GET'])
def product_list(request):
    products = Product.objects.filter(is_active=True)
    serializer = ProductSerializer(products, many=True)
    return Response(serializer.data)
```

```javascript
// Frontend: React component consuming the API
import { useState, useEffect } from 'react';

function ProductList() {
    const [products, setProducts] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        fetch('/api/v1/products/')
            .then(res => res.json())
            .then(data => {
                setProducts(data.results);
                setLoading(false);
            });
    }, []);

    if (loading) return <div>Loading...</div>;

    return (
        <div>
            {products.map(product => (
                <div key={product.id}>
                    <h3>{product.name}</h3>
                    <p>Rs. {product.price}</p>
                </div>
            ))}
        </div>
    );
}
```

Modern Architecture Trends

JAMstack (JavaScript, APIs, Markup):

- Pre-built static HTML + JavaScript + APIs
- No traditional server rendering at request time
- Fast, secure, scalable

- Tools: Next.js, Gatsby, Vercel

Serverless Architecture:

- Run code in functions without managing servers
- Scales automatically, pay per execution
- Examples: AWS Lambda, Vercel Functions, Netlify Functions
- Limitation: Cold start latency, not suitable for long-running tasks

Microservices:

- Application split into independent services
- Each service has own codebase, database, deployment
- Communicate via REST APIs or message queues

---

2. Testing and Quality Assurance

Why Testing is Critical

- Catches bugs before they reach production
- Prevents regression (fixing one thing breaking another)
- Documents expected behavior
- Enables confident refactoring and deployment
- Reduces cost — bugs are cheaper to fix early than in production

Types of Testing

1. Unit Testing

Tests the smallest individual units (functions, methods, classes) in isolation from all dependencies.

Characteristics:

- Fast (milliseconds per test)
- Isolated (mock/stub all external dependencies)
- Tests one thing at a time
- Should be run frequently during development

AAA Pattern (Arrange-Act-Assert):

```
# Django unit test example
from django.test import TestCase
from .models import Product
from .services import calculate_discount

class DiscountCalculatorTest(TestCase):
    def test_percentage_discount(self):
        # Arrange
        original_price = 1000
        discount_percent = 20

        # Act
        discounted_price = calculate_discount(original_price, discount_percent)

        # Assert
```

```
        self.assertEqual(discounted_price, 800)

    def test_zero_discount_returns_original_price(self):
        # Arrange
        original_price = 500

        # Act
        result = calculate_discount(original_price, 0)

        # Assert
        self.assertEqual(result, 500)

    def test_invalid_discount_raises_error(self):
        with self.assertRaises(ValueError):
            calculate_discount(100, -10)  # Negative discount not allowed
```

2. Integration Testing

Tests how multiple components or systems work TOGETHER. Tests interactions between modules.

What it tests:

- API endpoint + database
- Service + external API
- Frontend + backend API

```
# Integration test: API endpoint + database
from rest_framework.test import APITestCase
from rest_framework import status
from django.contrib.auth.models import User
from .models import Product, Category

class ProductAPIIntegrationTest(APITestCase):
    def setUp(self):
        self.user = User.objects.create_user(username='testuser', password='pass123')
        self.client.force_authenticate(user=self.user)
        self.category = Category.objects.create(name='Electronics')

    def test_create_and_retrieve_product(self):
        # Create via API
        create_data = {
            'name': 'iPhone 15',
            'price': '150000.00',
            'category': self.category.id,
            'stock': 10
        }
        create_response = self.client.post('/api/v1/products/', create_data)
        self.assertEqual(create_response.status_code, status.HTTP_201_CREATED)
        product_id = create_response.data['id']

        # Retrieve and verify it was actually saved to DB
        get_response = self.client.get(f'/api/v1/products/{product_id}/')
```

```
        self.assertEqual(get_response.status_code, status.HTTP_200_OK)
        self.assertEqual(get_response.data['name'], 'iPhone 15')

        # Verify in database directly
        product = Product.objects.get(pk=product_id)
        self.assertEqual(product.price, 150000.00)
```

3. System Testing

Tests the complete, integrated application as a whole against the functional requirements.

- Full end-to-end testing of all features
- Performed in a staging environment similar to production
- Validates business workflows work end-to-end

4. End-to-End (E2E) Testing

Tests user workflows through the actual browser interface — simulates real user interactions.

```
# Selenium E2E test example
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class CheckoutE2ETest:
    def setUp(self):
        self.driver = webdriver.Chrome()
        self.wait = WebDriverWait(self.driver, 10)

    def test_complete_purchase_flow(self):
        driver = self.driver

        # 1. Navigate to shop
        driver.get("https://shop.example.com")

        # 2. Search for product
        search_box = driver.find_element(By.NAME, "search")
        search_box.send_keys("Laptop")
        search_box.submit()

        # 3. Click first product
        self.wait.until(EC.presence_of_element_located((By.CLASS_NAME, "product-card")))
        driver.find_element(By.CLASS_NAME, "product-card").click()

        # 4. Add to cart
        driver.find_element(By.ID, "add-to-cart-btn").click()

        # 5. Go to checkout
        driver.find_element(By.ID, "checkout-btn").click()

        # 6. Verify order confirmation
        confirmation = self.wait.until(
```

```
            EC.presence_of_element_located((By.CLASS_NAME, "order-confirmed"))
        )
        self.assertIn("Order Confirmed", confirmation.text)

    def tearDown(self):
        self.driver.quit()
```

E2E Testing Tools: Selenium, Playwright, Cypress, Puppeteer

5. Manual Testing

Human tester explores the application without automated scripts.

- **Exploratory testing:** Tester freely explores the app
- **Ad-hoc testing:** Random testing without formal test cases
- Good for: UX evaluation, edge cases, creative scenarios

6. Automated Testing

Test scripts run automatically, often as part of CI/CD pipeline.

- **Faster** than manual for repetitive tests
- **Reproducible:** Same tests run identically every time
- **Regression testing:** Automatically verify no existing features broke

7. Performance Testing

Tests how the application behaves under load.

- **Load Testing:** Expected normal load (100 users)
- **Stress Testing:** Beyond expected load (1000 users) — find breaking point
- **Spike Testing:** Sudden traffic surge (sale event)
- **Soak Testing:** Extended duration at normal load (memory leaks)

Tools: Apache JMeter, Locust (Python), k6

```
# Locust performance test
from locust import HttpUser, task, between

class WebsiteUser(HttpUser):
    wait_time = between(1, 3)  # Random wait 1-3 seconds between requests

    @task(3)
    def view_products(self):
        self.client.get("/api/v1/products/")

    @task(1)
    def search_products(self):
        self.client.get("/api/v1/products/?search=laptop")

    @task(1)
    def view_product_detail(self):
        self.client.get("/api/v1/products/1/")
```

8. Security Testing

Tests the application for vulnerabilities.

- **Penetration Testing (Pen Test):** Authorized attempt to hack the application
- **SAST (Static Application Security Testing):** Analyze source code for vulnerabilities
- **DAST (Dynamic Application Security Testing):** Test running application (OWASP ZAP)
- **Vulnerability Scanning:** Automated tools scan for known vulnerabilities

Testing Pyramid

```
        /\
       /E2E\         ← Few, slow, expensive (10%)
      /──────\
     / Integr.\      ← Some, moderate speed (20%)
    /──────────\
   /    Unit    \    ← Many, fast, cheap (70%)
  /______________\
```

Principle: Write many unit tests (cheap, fast), fewer integration tests, and few E2E tests (slow, complex).

QA (Quality Assurance) vs Testing

Aspect	QA (Quality Assurance)	Testing
Focus	Process improvement	Finding defects
Approach	Preventive	Reactive
Scope	Entire SDLC	Testing phase
Goal	Prevent defects from occurring	Detect defects that exist
Activities	Reviews, process audits, standards	Test execution, bug reporting

QA Activities:

- Code reviews
- Establishing coding standards
- Reviewing requirements for completeness
- Defect prevention processes
- Metrics and reporting

Bug/Defect Lifecycle

```
New → Assigned → In Progress → Fixed → Testing → Verified → Closed
                                  ↓
                              Reopened (if fix doesn't work)
```

Test-Driven Development (TDD)

Write tests BEFORE writing the production code.

```
Red → Green → Refactor cycle:
1. RED:    Write a failing test for new functionality
2. GREEN:  Write minimum code to make test pass
3. REFACTOR: Improve code quality while keeping tests green
```

```python
# TDD Example: Password strength validator

# Step 1: Write the failing test first
class TestPasswordValidator(TestCase):
    def test_password_must_be_at_least_8_chars(self):
        with self.assertRaises(ValidationError):
            validate_password("short")

    def test_strong_password_passes(self):
        self.assertTrue(validate_password("StrongPass123!"))

# Step 2: Write the minimum code to make it pass
def validate_password(password):
    if len(password) < 8:
        raise ValidationError("Password must be at least 8 characters")
    return True

# Step 3: Refactor (add more rules, better structure)
```

---

3. DevOps and CI/CD

What is DevOps?

DevOps is a culture and set of practices that combines software development (Dev) and IT operations (Ops) to shorten the systems development lifecycle and provide continuous delivery with high quality.

```
Development → Testing → Staging → Production
    ↑                                  |
    └──────── Feedback Loop ────────────┘
```

DevOps Key Practices:

1. **Continuous Integration (CI)** — Merge code frequently, automated testing
2. **Continuous Delivery (CD)** — Always have deployable code
3. **Continuous Deployment (CD)** — Automatically deploy to production
4. **Infrastructure as Code (IaC)** — Manage infrastructure with code (Terraform, Ansible)
5. **Monitoring and Logging** — Observe production systems (Prometheus, Grafana, ELK)
6. **Automated Testing** — Tests run in pipeline, not just manually
7. **Containerization** — Docker for consistent environments

CI vs CD vs CD (Continuous Deployment)

Concept	Definition	Automation Level
Continuous Integration (CI)	Developers merge code frequently; automated build and test on every merge	Build + Test automated

Continuous Delivery	Code is always in a deployable state; deployment is manual trigger	Build + Test + Stage automated; deploy is manual
Continuous Deployment	Every passing build is automatically deployed to production	Everything automated, including production deploy

```
Code commit → [CI: Build + Test] → [CD: Stage Deploy] → [Manual approval] → [Production Deploy]
                                                                                ↓ (Continuous Deployment)
                                                                          [Auto Production Deploy]
```

CI/CD Pipeline Steps

A typical CI/CD pipeline:

```
1. Developer pushes code to Git repository
2. CI server detects push (webhook)
3. CI Pipeline begins:
   a. Checkout code
   b. Install dependencies
   c. Run linter (code style check)
   d. Run unit tests
   e. Run integration tests
   f. Build Docker image
   g. Push image to registry
4. CD Pipeline:
   a. Deploy to staging environment
   b. Run smoke tests on staging
   c. Manual approval (or auto for continuous deployment)
   d. Deploy to production
   e. Post-deployment health checks
```

GitHub Actions (CI/CD Example)

```yaml
# .github/workflows/django-ci.yml
name: Django CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    name: Run Tests
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:14
```

```yaml
      env:
        POSTGRES_DB: testdb
        POSTGRES_USER: postgres
        POSTGRES_PASSWORD: postgres
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Python 3.11
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt

    - name: Run linter (flake8)
      run: flake8 . --max-line-length=120 --exclude=migrations

    - name: Run Django tests
      env:
        DATABASE_URL: postgres://postgres:postgres@localhost:5432/testdb
        SECRET_KEY: test-secret-key-for-ci
        DEBUG: 'False'
      run: |
        python manage.py migrate
        python manage.py test --verbosity=2

    - name: Check test coverage
      run: |
        pip install coverage
        coverage run manage.py test
        coverage report --fail-under=80

deploy-staging:
  name: Deploy to Staging
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/develop'

  steps:
    - name: Deploy to staging
      run: |
        echo "Deploying to staging server..."
```

```
        # ssh deploy@staging.server.com 'cd /app && git pull && ./deploy.sh'

  deploy-production:
    name: Deploy to Production
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    environment: production  # Requires manual approval in GitHub

    steps:
      - name: Build and push Docker image
        run: |
          docker build -t myapp:${{ github.sha }} .
          docker tag myapp:${{ github.sha }} registry.example.com/myapp:latest
          docker push registry.example.com/myapp:latest

      - name: Deploy to production
        run: |
          echo "Deploying to production..."
```

---

4. Deployment Platforms and Docker

Docker

Docker is a platform for developing, shipping, and running applications in containers.

Why Docker?

- "Works on my machine" problem eliminated — containers behave identically everywhere
- Isolates application from host OS
- Easy to replicate exact production environment in development
- Fast startup (seconds vs minutes for VMs)

Key Docker Concepts:

Concept	Description
Image	Read-only template for creating containers (like a class)
Container	Running instance of an image (like an object)
Dockerfile	Instructions to build an image
Registry	Stores Docker images (Docker Hub, AWS ECR, GitHub Container Registry)
Volume	Persistent storage that survives container restarts
Network	Communication between containers

Dockerfile for Django:

```
# Dockerfile
FROM python:3.11-slim
```

```
# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Set work directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    libpq-dev \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy project files
COPY . .

# Collect static files
RUN python manage.py collectstatic --noinput

# Create non-root user for security
RUN useradd -m appuser && chown -R appuser /app
USER appuser

# Expose port
EXPOSE 8000

# Start Gunicorn server
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "3", "myproject.wsgi:application"]
```

docker-compose.yml (Multi-Container Setup):

```
# docker-compose.yml
version: '3.8'

services:
  db:
    image: postgres:14
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      POSTGRES_DB: myapp_db
      POSTGRES_USER: myapp_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    restart: unless-stopped

  redis:
```

```
    image: redis:7-alpine
    restart: unless-stopped

  web:
    build: .
    command: gunicorn myproject.wsgi:application --bind 0.0.0.0:8000 --workers 3
    volumes:
      - static_files:/app/staticfiles
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - db
      - redis
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
      - static_files:/app/staticfiles
    ports:
      - "80:80"
      - "443:443"
    depends_on:
      - web
    restart: unless-stopped

  celery:
    build: .
    command: celery -A myproject worker -l info
    env_file:
      - .env
    depends_on:
      - db
      - redis
    restart: unless-stopped

volumes:
  postgres_data:
  static_files:
```

Basic Docker Commands:

```
# Build image
docker build -t myapp:latest .

# Run container
docker run -p 8000:8000 --env-file .env myapp:latest
```

```
# Docker Compose
docker-compose up -d          # Start all services in background
docker-compose down           # Stop all services
docker-compose logs -f web    # Follow logs for web service
docker-compose exec web bash  # Open shell in running container

# List running containers
docker ps

# Stop and remove everything
docker-compose down -v        # Also removes volumes
```

Deployment Platforms

Platform	Type	Best For	Cost
Heroku	PaaS	Quick deployment, startups	Free tier available
Render	PaaS	Modern Heroku alternative	Free tier
Railway	PaaS	Developer-friendly	Free tier
DigitalOcean	VPS/PaaS	Control + ease	$4/month+
AWS (EC2, ECS, Elastic Beanstalk)	IaaS/PaaS	Enterprise, scalability	Pay as you go
Google Cloud Run	Serverless containers	Scalable apps	Pay per request
Vercel	PaaS (frontend)	Next.js, static sites	Free tier
Netlify	PaaS (frontend)	JAMstack, static sites	Free tier

Production Django Deployment Checklist

```
# settings/production.py
DEBUG = False
ALLOWED_HOSTS = ['myapp.com', 'www.myapp.com']

# Database (use PostgreSQL in production, not SQLite)
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ['DB_NAME'],
        'USER': os.environ['DB_USER'],
        'PASSWORD': os.environ['DB_PASSWORD'],
        'HOST': os.environ['DB_HOST'],
        'PORT': '5432',
    }
}
```

```
# Security
SECRET_KEY = os.environ['DJANGO_SECRET_KEY']
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

# Static files (use CDN/S3 in production)
STATIC_ROOT = '/var/www/myapp/static/'
STATICFILES_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
```

---

5. Progressive Web Applications (PWA)

What is a PWA?

A **Progressive Web Application (PWA)** is a web app that uses modern web technologies to deliver app-like experiences to users. PWAs are reliable, fast, and engaging.

Key Characteristics:

- Works offline or on poor network connections
- Installable on device home screen
- Receives push notifications
- Feels like a native app
- Responsive on all screen sizes

PWA Core Technologies

Service Workers

A **service worker** is a JavaScript file that runs in the background, separate from the web page. It acts as a programmable network proxy.

What Service Workers Enable:

- Offline functionality (cache resources)
- Background sync
- Push notifications
- Background data fetching

Service Worker Registration:

```
// index.js (main app file)
if ('serviceWorker' in navigator) {
    window.addEventListener('load', () => {
        navigator.serviceWorker.register('/sw.js')
            .then(registration => {
                console.log('SW registered:', registration.scope);
            })
            .catch(error => {
                console.log('SW registration failed:', error);
            });
```

```
    });
}
```

Service Worker File (sw.js):

```
const CACHE_NAME = 'myapp-v1';
const ASSETS_TO_CACHE = [
    '/',
    '/index.html',
    '/css/app.css',
    '/js/app.js',
    '/images/logo.png',
    '/offline.html'
];

// Install event: Cache essential assets
self.addEventListener('install', (event) => {
    event.waitUntil(
        caches.open(CACHE_NAME).then((cache) => {
            console.log('Caching app shell');
            return cache.addAll(ASSETS_TO_CACHE);
        })
    );
    self.skipWaiting();
});

// Activate event: Clean up old caches
self.addEventListener('activate', (event) => {
    event.waitUntil(
        caches.keys().then((cacheNames) => {
            return Promise.all(
                cacheNames
                    .filter(name => name !== CACHE_NAME)
                    .map(name => caches.delete(name))
            );
        })
    );
});

// Fetch event: Serve from cache or network
self.addEventListener('fetch', (event) => {
    event.respondWith(
        // Cache-first strategy for assets
        caches.match(event.request).then((cachedResponse) => {
            if (cachedResponse) {
                return cachedResponse;  // Serve from cache
            }
            // Not in cache — fetch from network
            return fetch(event.request).catch(() => {
                // Network failed — serve offline page
                return caches.match('/offline.html');
```

```
            });
        })
    );
});
```

Caching Strategies:

Strategy	Description	Use Case
Cache First	Serve cache, fallback to network	Static assets (CSS, JS, images)
Network First	Try network, fallback to cache	API data that needs to be fresh
Stale While Revalidate	Serve cache immediately, update in background	News feeds, product listings
Cache Only	Only serve from cache	App shell, offline page
Network Only	Always fetch from network	Real-time data

Web App Manifest

A JSON file that tells the browser how your app should behave when installed.

```
// manifest.json (linked from index.html)
{
  "name": "My Shopping App",
  "short_name": "ShopApp",
  "description": "The best shopping experience on any device",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#2196F3",
  "orientation": "any",
  "icons": [
    {
      "src": "/icons/icon-72x72.png",
      "sizes": "72x72",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable"
    },
    {
      "src": "/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ],
  "shortcuts": [
    {
```

```
      "name": "My Orders",
      "url": "/orders",
      "icons": [{"src": "/icons/orders.png", "sizes": "192x192"}]
    }
  ]
}
```

```
<!-- Link manifest in index.html -->
<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#2196F3">
<meta name="apple-mobile-web-app-capable" content="yes">
```

PWA Display Modes

- `standalone` : App looks like native (no browser UI, own window)
- `fullscreen` : Full screen, no system UI
- `minimal-ui` : Browser with minimal navigation
- `browser` : Regular browser tab

PWA Advantages

1. No app store submission required — instant updates
2. Works offline with service worker caching
3. Installable directly from the browser
4. Single codebase for all platforms (web, Android, iOS)
5. Smaller size than native apps
6. Discoverable via search engines
7. Can send push notifications
8. Progressive enhancement — works for all browsers

PWA Limitations

1. **iOS limitations:** Push notifications and background sync limited on iOS/Safari
2. **Hardware access:** No access to contacts, call logs, NFC (limited hardware APIs vs native)
3. **No app store visibility:** Cannot be found in Play Store/App Store organically
4. **Storage limits:** Service worker cache has browser-imposed limits
5. **Performance:** Still not as performant as native for graphics-intensive apps

PWA Requirements Checklist

- ☐ Served over HTTPS
- ☐ Has a Web App Manifest
- ☐ Has a registered Service Worker
- ☐ Has icons (at least 192x192 and 512x512)
- ☐ `start_url` loads while offline
- ☐ Has a theme color
- ☐ Redirects HTTP → HTTPS

---

6. Responsive Web Design

What is Responsive Design?

Responsive Web Design creates web pages that look and work well on all devices and screen sizes — desktops (1200px+), tablets (768px), and mobiles (320px-480px) — using a single HTML/CSS codebase.

Core Principles

1. Flexible Grid Layout

CSS Flexbox:

```
/* Navigation that wraps on small screens */
.navbar {
    display: flex;
    flex-wrap: wrap;          /* Wrap items when space runs out */
    justify-content: space-between;
    align-items: center;
    gap: 1rem;
}

/* Card grid */
.product-grid {
    display: flex;
    flex-wrap: wrap;
    gap: 1.5rem;
}
.product-card {
    flex: 1 1 280px;  /* Grow, shrink, base width 280px */
    max-width: 400px;
}
```

CSS Grid:

```
/* Responsive grid with auto columns */
.product-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
    gap: 1.5rem;
}

/* 3-column on desktop, 2 on tablet, 1 on mobile */
.layout {
    display: grid;
    grid-template-columns: 1fr 3fr 1fr;  /* sidebar | main | sidebar */
}
```

2. Media Queries

Media queries apply CSS rules based on device characteristics.

```
/* Mobile-first approach: base styles for mobile, override for larger screens */

/* Base styles (mobile – small screens first) */
.container {
```

```
    width: 100%;
    padding: 0 1rem;
}

.product-grid {
    display: grid;
    grid-template-columns: 1fr;  /* 1 column on mobile */
    gap: 1rem;
}

/* Tablet (768px and up) */
@media (min-width: 768px) {
    .container {
        max-width: 768px;
        margin: 0 auto;
    }

    .product-grid {
        grid-template-columns: repeat(2, 1fr);  /* 2 columns */
    }

    .sidebar {
        display: block;  /* Show sidebar on tablet */
    }
}

/* Desktop (1024px and up) */
@media (min-width: 1024px) {
    .container {
        max-width: 1200px;
    }

    .product-grid {
        grid-template-columns: repeat(3, 1fr);  /* 3 columns */
    }

    .hero-section {
        font-size: 3rem;  /* Larger text on desktop */
    }
}

/* Large Desktop (1440px and up) */
@media (min-width: 1440px) {
    .product-grid {
        grid-template-columns: repeat(4, 1fr);  /* 4 columns */
    }
}

/* Orientation-based media query */
@media (orientation: landscape) and (max-width: 768px) {
    .mobile-menu { display: none; }
}
```

```css
/* Print styles */
@media print {
    .navbar, .sidebar, .ads { display: none; }
    body { font-size: 12pt; }
}
```

Common Breakpoints:

Device	Breakpoint
Mobile (small)	320px - 480px
Mobile (large)	481px - 767px
Tablet	768px - 1023px
Desktop	1024px - 1279px
Large Desktop	1280px+

3. Responsive Images

```html
<!-- Responsive image with srcset -->
<img
    src="product-small.jpg"
    srcset="
        product-small.jpg  480w,
        product-medium.jpg 768w,
        product-large.jpg  1200w
    "
    sizes="
        (max-width: 480px) 100vw,
        (max-width: 768px) 50vw,
        33vw
    "
    alt="Product photo"
    loading="lazy"
>

<!-- Art direction: different image on different screens -->
<picture>
    <source media="(max-width: 480px)" srcset="product-portrait.jpg">
    <source media="(min-width: 481px)" srcset="product-landscape.jpg">
    <img src="product-landscape.jpg" alt="Product">
</picture>
```

```css
/* Make all images responsive by default */
img {
    max-width: 100%;  /* Never wider than container */
```

```
    height: auto;     /* Maintain aspect ratio */
}
```

4. Viewport Meta Tag (Critical for Mobile)

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without this, mobile browsers zoom out to show the full desktop width.

5. Mobile-First Design Philosophy

Design for the smallest screen first, then progressively enhance for larger screens.

Why mobile-first:

- More than 60% of web traffic is from mobile devices
- Forces prioritization of essential content
- `min-width` media queries are more performant than `max-width`
- Better SEO (Google uses mobile-first indexing)

```
/* Mobile-first (RECOMMENDED): Start with mobile, add complexity for large screens */
.menu { display: none; }            /* Hidden on mobile */
@media (min-width: 768px) {
    .menu { display: flex; }        /* Visible on tablet+ */
}

/* Desktop-first (NOT recommended): Start with desktop, remove for small screens */
.menu { display: flex; }            /* Starts visible */
@media (max-width: 767px) {
    .menu { display: none; }        /* Hide on mobile */
}
```

6. Relative Units

```
/* Avoid fixed px widths — use relative units */
.container {
    width: 90%;           /* Percentage of parent */
    max-width: 1200px;    /* Cap at 1200px */
    font-size: 1rem;      /* rem = relative to root element (16px default) */
    padding: 2em;         /* em = relative to element's font-size */
    height: 50vh;         /* vh = viewport height */
    width: 100vw;         /* vw = viewport width */
}
```

Responsive Design with Bootstrap (Framework)

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
    <div class="container">
        <div class="row">
            <!-- col-12 on mobile, col-md-6 on tablet, col-lg-4 on desktop -->
            <div class="col-12 col-md-6 col-lg-4">
                <div class="card">Product 1</div>
            </div>
            <div class="col-12 col-md-6 col-lg-4">
                <div class="card">Product 2</div>
            </div>
            <div class="col-12 col-md-6 col-lg-4">
                <div class="card">Product 3</div>
            </div>
        </div>
    </div>
</body>
</html>
```

---

7. Usability Principles

What is Usability?

Usability is the ease with which users can learn and use a product to achieve their goals. A usable website is intuitive, efficient, and satisfying.

Jakob Nielsen's 10 Usability Heuristics

1. **Visibility of System Status**
    - Users should always know what's happening
    - Show loading spinners, progress bars, success/error messages
    - Example: "Your order is being processed..." with a spinner
2. **Match Between System and Real World**
    - Use familiar language and concepts (not technical jargon)
    - Use familiar icons (shopping cart, magnifying glass for search)
    - Example: "Delete" not "Terminate object record"
3. **User Control and Freedom**
    - Provide undo/redo, cancel buttons
    - Allow users to exit unwanted states
    - Example: "Undo" after accidentally deleting a file
4. **Consistency and Standards**
    - Follow platform conventions (blue underlined text = link)
    - Consistent terminology, layout, colors throughout the app
    - Example: The search bar is always at the top right

5. **Error Prevention**
    - Design to prevent errors from occurring in the first place
    - Confirm dangerous actions ("Are you sure you want to delete?")
    - Example: Graying out a "Submit" button until all required fields are filled
6. **Recognition Rather Than Recall**
    - Minimize memory load — show options rather than requiring typing
    - Use autocomplete, dropdown menus, visible labels
    - Example: Search suggestions as you type
7. **Flexibility and Efficiency of Use**
    - Provide shortcuts for expert users while remaining simple for novices
    - Keyboard shortcuts, customizable layouts
    - Example: Ctrl+K for quick search in VS Code
8. **Aesthetic and Minimalist Design**
    - Don't clutter interfaces with irrelevant information
    - Every extra element competes for attention and reduces usability
    - Example: Google's homepage — just a search box
9. **Help Users Recognize, Diagnose, and Recover from Errors**
    - Error messages should be in plain language (not error codes)
    - Suggest constructive solutions
    - Example: "Password must be at least 8 characters" (not "Error 422")
10. **Help and Documentation**
    - Provide easily searchable help documentation
    - Context-sensitive help (help icons near complex fields)
    - Example: "?" tooltip explaining what a field expects

Additional Usability Factors

Accessibility (a11y):

```
<!-- Alt text for screen readers -->
<img src="logo.png" alt="Company Logo">

<!-- Proper form labels -->
<label for="email">Email Address</label>
<input type="email" id="email" name="email" required aria-describedby="email-hint">
<span id="email-hint">We'll never share your email with anyone.</span>

<!-- ARIA roles and attributes -->
<nav aria-label="Main navigation">
<button aria-label="Close dialog">✕</button>
<div role="alert" aria-live="polite">Form submitted successfully!</div>
```

Performance as Usability:

- Google: 53% of mobile users abandon a site that takes >3 seconds to load

- Core Web Vitals:
  - **LCP (Largest Contentful Paint):** < 2.5 seconds
  - **FID (First Input Delay):** < 100ms
  - **CLS (Cumulative Layout Shift):** < 0.1

---

8. Modern Web Architecture Trends

JAMstack

JAMstack = **J**avaScript + **A**PIs + **M**arkup

```
Traditional Web:
Client → Server → Database → Render HTML → Client

JAMstack:
Client ← Pre-built HTML (CDN)
Client → APIs (headless CMS, third-party APIs)
```

Advantages:

- Better performance (pre-built HTML served from CDN)
- Better security (no server-side code to attack)
- Lower cost (static hosting is cheap)
- Better developer experience

Tools: Next.js, Gatsby, Hugo, Netlify, Vercel

Serverless Computing

Run code without managing servers. Functions are event-driven and auto-scale.

```python
# AWS Lambda function (serverless)
def lambda_handler(event, context):
    """Triggered by API Gateway, SQS, S3 events, etc."""
    user_id = event['pathParameters']['userId']
    # ... process request ...
    return {
        'statusCode': 200,
        'body': json.dumps({'user': user_data})
    }
```

When to use: Event-driven tasks, infrequent jobs, microservices. **When NOT to use:** Long-running processes, high-frequency requests (cold starts).

Containerization vs Virtualization

Aspect	Virtual Machine	Docker Container
OS	Full OS per VM	Shares host OS kernel
Size	GBs	MBs

Startup	Minutes	Seconds
Isolation	Full hardware isolation	Process-level isolation
Performance	Overhead from hypervisor	Near-native
Use case	Full OS isolation needed	App isolation, microservices

---

9. Model Exam Questions & Answers

Short Answer Questions

Q1. What is a Progressive Web Application (PWA)? List its key features and advantages. [2075, 2077, 2079]

Answer:

A **Progressive Web Application (PWA)** is a web application that uses modern browser technologies (service workers, web app manifest, HTTPS) to deliver native app-like experiences — including offline functionality, push notifications, and home screen installation — through a standard browser.

Key Features:

- **Offline capability:** Works without internet via service worker caching
- **Installable:** Can be added to home screen from browser (no app store)
- **Push notifications:** Can notify users even when the app isn't open
- **Responsive:** Works on all screen sizes
- **Fast:** Pre-cached resources load instantly
- **Secure:** Must be served over HTTPS

Advantages:

1. No app store submission — instant updates, no approval process
2. Single codebase for all platforms
3. Smaller size than native apps
4. Discoverable via search engines
5. Works offline or on poor connections
6. Can send push notifications
7. Lower development cost than native apps

Q2. Explain the role of Service Workers in PWAs. [2076, 2078, 2080]

Answer:

A **service worker** is a JavaScript file that runs in the background in a separate thread from the main browser thread. It acts as a programmable proxy between the browser and the network.

Key Capabilities:

1. **Offline caching:** Intercepts network requests and serves cached responses when offline
2. **Background sync:** Queues failed requests and retries when connectivity returns
3. **Push notifications:** Receives and displays notifications even when the app is closed
4. **Cache management:** Pre-caches app shell during install for instant load

Service Worker Lifecycle:

```
Install → Activate → Fetch (intercept requests)
```

Caching Strategies:

- **Cache First:** Serve cached response; update cache in background (good for static assets)
- **Network First:** Try network; fall back to cache if offline (good for fresh data)
- **Stale While Revalidate:** Serve cache immediately; update in background (best UX balance)

Example:

```
// Install: cache app shell
self.addEventListener('install', event => {
    event.waitUntil(
        caches.open('v1').then(cache => cache.addAll(['/index.html', '/app.css']))
    );
});

// Fetch: serve from cache or network
self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request).then(cached => cached || fetch(event.request))
    );
});
```

Q3. What is CI/CD? Explain its stages and benefits. [2074, 2076, 2078]

Answer:

CI/CD (Continuous Integration/Continuous Delivery) is a DevOps practice that automates the software build, test, and deployment pipeline.

CI (Continuous Integration):

- Developers merge code to shared repository frequently (multiple times/day)
- Every merge triggers automated build and tests
- Goal: Detect integration problems early

CD (Continuous Delivery):

- Code is always in a deployable state
- Deployment to production requires a manual trigger
- Automated staging deployment

CD (Continuous Deployment):

- Every code change that passes all tests is automatically deployed to production
- No manual approval needed

CI/CD Pipeline Stages:

1. Code commit to Git
2. Automated build (compile, package)
3. Static analysis / linting
4. Unit tests
5. Integration tests

6. Build Docker image
7. Deploy to staging
8. Smoke tests on staging
9. Manual approval (for CD, auto for Continuous Deployment)
10. Deploy to production
11. Post-deployment monitoring

Benefits:

- Faster release cycles
- Early bug detection (cheaper to fix)
- Reduced manual errors in deployment
- Always deployable codebase
- Automated testing coverage
- Rapid feedback to developers

Q4. Explain the types of software testing with examples. [2072, 2073, 2077]

Answer:

1. Unit Testing: Tests individual functions/methods in isolation.

```
def test_calculate_tax():
    assert calculate_tax(1000, 0.13) == 130  # 13% tax
```

2. Integration Testing: Tests how multiple components work together.

- Test: API endpoint + database — verify that creating a product via API actually saves to DB

3. System Testing: Tests the entire application as a whole against requirements.

- Full end-to-end functionality in a staging environment

4. E2E Testing: Tests user flows through the browser interface.

- Selenium: Visit site → search product → add to cart → checkout → verify order confirmation

5. Performance Testing: Tests behavior under load.

- Load test: 100 concurrent users on the product listing page
- Stress test: 10,000 users — at what point does it crash?

6. Security Testing: Tests for vulnerabilities.

- OWASP ZAP scan for XSS, SQL injection vulnerabilities

7. Manual Testing: Human tester explores the app without scripts.

- Exploratory testing to find unexpected UX issues

Testing Pyramid: Many unit tests (fast, cheap) → fewer integration tests → few E2E tests (slow, complex). This gives maximum coverage with minimum time cost.

Q5. What is Docker? Explain its key concepts and how it's used in web application deployment. [2075, 2078, 2079]

Answer:

Docker is an open-source platform for developing, shipping, and running applications in isolated containers.

Problem Docker Solves: "It works on my machine" — Docker ensures the application runs identically in development, testing, and production.

Key Concepts:

Concept	Description
Image	Read-only template for creating containers (like a class/blueprint)
Container	A running instance of an image (like an object instance)
Dockerfile	Script of instructions to build a Docker image
Docker Hub	Public registry for storing and sharing Docker images
Volume	Persistent storage that survives container restarts
Docker Compose	Tool for defining and running multi-container applications

Dockerfile for a Django App:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "myproject.wsgi:application", "--bind", "0.0.0.0:8000"]
```

Benefits:

- Consistent environments across dev/staging/production
- Isolation — each container has its own dependencies
- Easy scalability with container orchestration (Kubernetes)
- Fast deployment — images are pre-built
- Microservices-friendly — each service in its own container

Production Stack with Docker Compose:

- Django container (app server)
- PostgreSQL container (database)
- Redis container (cache/sessions)
- Nginx container (reverse proxy/static files)

Q6. Explain responsive web design. What are media queries and how are they used? [2071, 2073, 2076]

Answer:

Responsive Web Design creates web pages that adapt their layout and content to provide an optimal viewing experience on all devices (mobile, tablet, desktop) using a single HTML/CSS codebase.

Core Techniques:

1. Flexible grid layouts (CSS Flexbox/Grid)

2. Media queries
3. Responsive images (`srcset` , `max-width: 100%`)
4. Viewport meta tag (`<meta name="viewport" content="width=device-width, initial-scale=1">`)
5. Relative units (%, rem, vh, vw)

Media Queries: CSS rules that apply styles based on device/screen characteristics.

Syntax:

```
@media (condition) { CSS rules }
```

Example — Mobile-First Responsive Layout:

```
/* Mobile (base) */
.product-grid {
    display: grid;
    grid-template-columns: 1fr;  /* 1 column */
}

/* Tablet (768px+) */
@media (min-width: 768px) {
    .product-grid {
        grid-template-columns: repeat(2, 1fr);  /* 2 columns */
    }
}

/* Desktop (1024px+) */
@media (min-width: 1024px) {
    .product-grid {
        grid-template-columns: repeat(3, 1fr);  /* 3 columns */
    }
}
```

Mobile-First Approach: Design for mobile screens first, then use `min-width` media queries to add complexity for larger screens. This is preferred because most web traffic is from mobile, and it encourages content prioritization.

Common Breakpoints: 480px (mobile), 768px (tablet), 1024px (desktop), 1280px (large desktop).

Q7. What is DevOps? How does it bridge development and operations? Explain the CI/CD pipeline with a GitHub Actions example. [2077, 2079, 2080]

Answer:

DevOps is a culture and practice that unifies software development (Dev) and IT operations (Ops) to improve collaboration, automate processes, and deliver software faster and more reliably.

DevOps Key Practices:

- Continuous Integration and Delivery (CI/CD)
- Infrastructure as Code (manage servers via code)
- Automated testing
- Monitoring and logging
- Containerization (Docker, Kubernetes)

- Shift-left security (test security early)

How DevOps Bridges Dev and Ops:

- Developers write code with deployment in mind
- Operations team involved in architecture decisions
- Shared responsibility for production stability
- Automated pipeline from code to production
- Fast feedback loop via monitoring

GitHub Actions CI/CD Pipeline:

```yaml
name: Django CI/CD

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with: {python-version: '3.11'}
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: python manage.py test
      - name: Run linter
        run: flake8 .

  deploy:
    needs: test   # Only deploy if tests pass
    runs-on: ubuntu-latest
    steps:
      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .
      - name: Push to registry
        run: docker push registry.example.com/myapp:latest
      - name: Deploy to server
        run: ssh deploy@prod.example.com 'docker-compose pull && docker-compose up -d'
```

Pipeline Flow:

```
git push → GitHub Actions triggered →
  [1. Checkout code]
  [2. Install dependencies]
  [3. Run linter]
  [4. Run unit tests]
  [5. Run integration tests]
```

```
  [6. Build Docker image]
  [7. Deploy to staging]
  [8. Deploy to production]
→ Slack notification: "Deployment successful!"
```

Q8. What is full-stack web development? Describe the frontend, backend, and database layers and how they integrate. [2071, 2072, 2074]

Answer:

Full-stack web development encompasses working on all three layers of a web application: the frontend (client-side), backend (server-side), and database.

1. Frontend (Presentation Layer):

- What the user sees and interacts with in the browser
- Technologies: HTML5 (structure), CSS3 (styling), JavaScript (interactivity)
- Frameworks: React, Vue, Angular
- Responsibilities: UI rendering, user input handling, calling backend APIs

2. Backend (Application Layer):

- Runs on the server; handles business logic
- Technologies: Django, Node.js, Flask, Spring Boot
- Responsibilities: Processing requests, authentication, authorization, calling the database, returning responses

3. Database (Data Layer):

- Stores and retrieves persistent data
- Relational: PostgreSQL, MySQL (structured data, ACID)
- NoSQL: MongoDB (documents), Redis (key-value cache)
- Responsibilities: Data storage, queries, transactions

Integration Flow (Modern SPA + REST API):

```
1. User opens browser → loads React app (HTML/CSS/JS bundle)
2. React app makes API call: GET /api/v1/products/
3. Django backend receives request, authenticates user via JWT
4. Django queries PostgreSQL: SELECT * FROM products
5. Django serializes results to JSON, returns 200 response
6. React renders products on screen
7. User clicks "Add to Cart" → POST /api/v1/cart/items/
8. Django validates, saves to database, returns 201 Created
9. React updates cart count in UI
```

This separation of frontend and backend (via REST API) allows:

- Independent deployment of frontend and backend
- Mobile apps can use the same backend API
- Different teams can work on frontend/backend simultaneously

---

End of Chapter 6 Notes

Total Chapter Coverage: Full-stack development (3 layers, technologies, integration), 8 testing types (Unit/AAA, Integration, System, E2E, Manual, Automated, Performance, Security), Testing Pyramid, TDD, QA vs Testing, Bug

lifecycle, DevOps principles, CI/CD (CI vs CD vs Continuous Deployment), GitHub Actions YAML pipeline, Docker (concepts, Dockerfile, docker-compose), Deployment platforms, PWA (service workers, web app manifest, caching strategies, advantages/limitations), Responsive design (Flexbox, Grid, media queries, mobile-first, breakpoints, responsive images), Usability heuristics (Nielsen's 10), Accessibility, Modern trends (JAMstack, Serverless, Containerization).